

Reviewer Pack — Minimal Index of Geometric Langlands Duals and Resolution Pr...

Entry eb40b656e3e8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 05:31:50 UTC

Minimal Index of Geometric Langlands Duals and Resolution Proof Width

Entry ID: eb40b656e3e8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 05:31:50 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given n -vertex graph G , the minimal index ($\text{ind}(G)$) of its associated geometric Langlands dual is linearly correlated with its resolution proof width $w(G)$, such that $\text{ind}(G) = \Omega(w(G))$. Furthermore, for all instances with property P , where P is defined as 'G has a geometric Langlands dual with index $\leq c \cdot n^{1/4}$ ', the following holds: $\text{ind}(G) \geq \Theta(c \cdot n^{1/4})$

Rationale (proposer's reasoning):

Geometric Langlands theory provides a bridge between algebraic geometry and representation theory, which could potentially offer new insights into complexity theory. The geometric Langlands dual is expected to capture essential properties of the graph that might influence its resolution proof width, suggesting a non-trivial lower bound.

Taxonomy category: Geometric_Langlands (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b3d7ed341f34808b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of generated n -vertex graphs with property P and resolution proof width $w(G)$, the index $\text{ind}(G)$ satisfies $\text{ind}(G) \geq c \cdot n^{1/4}$ AND $\text{ind}(G)/w(G) \leq 2.5$, where c is a constant ≤ 1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -geometric langlands theory AND resolution proof complexity - resolution proof width IN GEOMETRIC LANGLANDS THEORY - index of geometric langlands duals correlation with resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/2504.04416v1] Meta-Mathematics of Computational Complexity Theory - [http://arxiv.org/abs/2302.00039v1] Between Coherent and Constructible Local Langlands Correspondences - [http://arxiv.org/abs/hep-th/0512172v1] Lectures on the Langlands Program and Conformal Field Theory - [http://arxiv.org/abs/2107.01732v4] Gauge Theory and the Analytic Form of the Geometric Langlands Program - [http://arxiv.org/abs/2311.03743v2] A general framework for the analytic Langlands correspondence - [http://arxiv.org/abs/2008.02998v3] Coherent sheaves on the stack of Langlands parameters

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_random_graph(n):
    edges = set()
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < 0.5:
                edges.add((i, j))
    return edges

def resolution_width(graph):
    # Placeholder function to simulate resolution width calculation
    # Replace with actual DPLL solver implementation
    return len(graph) * 2

def geometric_langlands_index(graph):
    # Placeholder function to simulate geometric Langlands index calculation
    # Replace with actual Geometric Langlands theory implementation
    return len(graph)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    metric_name = "resolution_width"
```

```

instances_tested = 0
n_max = 0
ind_values = []
w_values = []
conjecture_holds = True
counterexample = ""

for n in [5, 10, 15, 20, 30, 40]:
    if time.time() + 8 < end_time:
        graph = generate_random_graph(n)
        w_G = resolution_width(graph)
        ind_G = geometric_langlands_index(graph)

        instances_tested += 1
        n_max = max(n_max, n)

        ind_values.append(ind_G)
        w_values.append(w_G)

        if len(ind_values) >= 30:
            break

mean_ind = sum(ind_values) / len(ind_values)
std_dev = math.sqrt(sum((x - mean_ind) ** 2 for x in ind_values) / len(ind_values))

if any(ind < c * n ** (1/4) for ind, n in zip(ind_values, [5, 10, 15, 20, 30, 40])):
    conjecture_holds = False
    counterexample = "ind(G) < c·n^(1/4)"

if any(ind / w > 2.5 for ind, w in zip(ind_values, w_values)):
    conjecture_holds = False
    counterexample = "ind(G)/w(G) > 2.5"

return {
    "metric_name": metric_name,
    "metric_value": mean_ind,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    import time

    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]

    end_time = time.time() + 240

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_ind = sum(r["metric_value"] for r in results) / len(results)
std_dev = math.sqrt(sum((r["metric_value"] - mean_ind) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ind} std={std_dev} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27b8952a.py", line 96, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27b8952a.py", line 65, in run_trial
    if any(ind < c * n ** (1/4) for ind, n in zip(ind_values, [5, 10, 15, 20, 30, 40])):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27b8952a.py", line 65, in <genexpr>
    if any(ind < c * n ** (1/4) for ind, n in zip(ind_values, [5, 10, 15, 20, 30, 40])):
    ^
NameError: name 'c' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's support or falsification. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18621
2	preregistration	ollama_remote	glm4:latest	0	0	9618
3	novelty	ollama_remote	glm4:latest	0	0	11368
4	novelty	ollama_remote	glm4:latest	0	0	14744
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26835
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14146
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11158
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17228
9	judge	ollama_remote	glm4:latest	0	0	11679

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 135397 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/eb40b656e3e8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eb40b656e3e8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eb40b656e3e8.tar.gz` (if generated)